MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

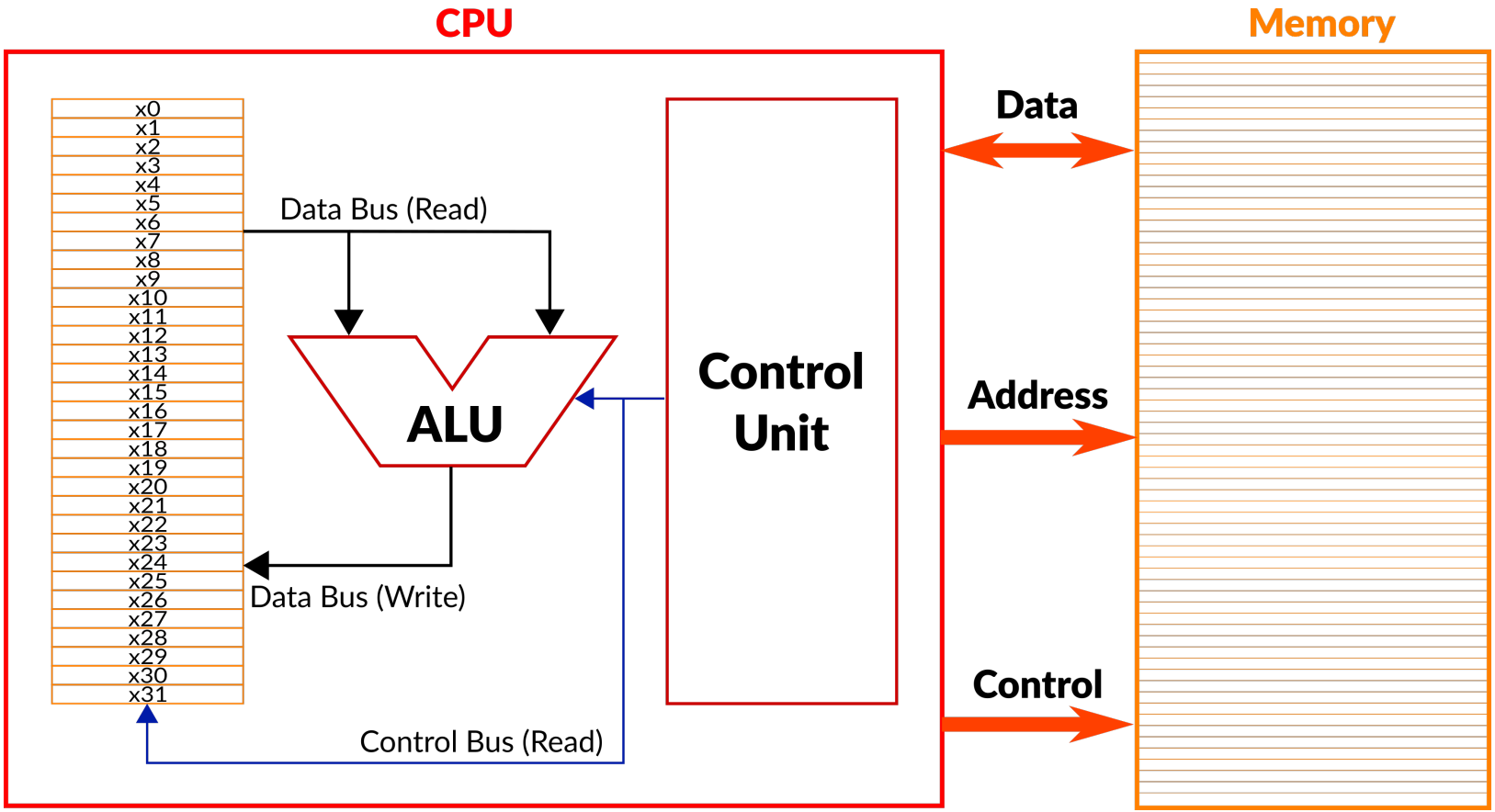
Integer Arithmetic

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- How to use assembly for integer arithmetic
- Trade-offs in how we can write the code
- Limitations in how we can write the code

RISC-V: The simplistic view



Memory Instructions

- `lw`: Load Word from memory
  - **Syntax:** `lw rd, imm[rs]`
- Pseudo `lw`: Load Word from label
  - **Syntax**: `lw rd, label`

- `sw`: Store Word to memory
  - **Syntax:** `sw rs1, imm[rs2]`
- Pseudo `sw`: Store Word to label
  - **Syntax**: `sw rs, label, td`

Processing Instructions

	Arithmetic	Logic	Shift	Comparison
Register only	`add` → Addition	`and` → Bitwise AND	`sll` → Shift left logical	
	`sub` → Subtraction	`or` → Bitwise OR	`srl` → Shift right logical	
	`mul{..}`→Multiply No option → lower half of 64-bit result h → upper half (signed) hu → upper half (unsigned) hsu → upper half (signed * unsigned)	`xor` → Bitwise XOR	`sra` → Shift right arithmetic	`slt{u}` → set if rs1 < rs2
	`div{u}` → Division (optional u indicates unsigned)			
	`rem{u}` → Remainder (optional u indicates unsigned)			
Immediate versions	`addi`	`andi`	`slli`	`slti{u}`
	~~`subi`~~ (pseudo)	`ori`	`srli`	
		`xori`	`srai`	

The Memory Wall

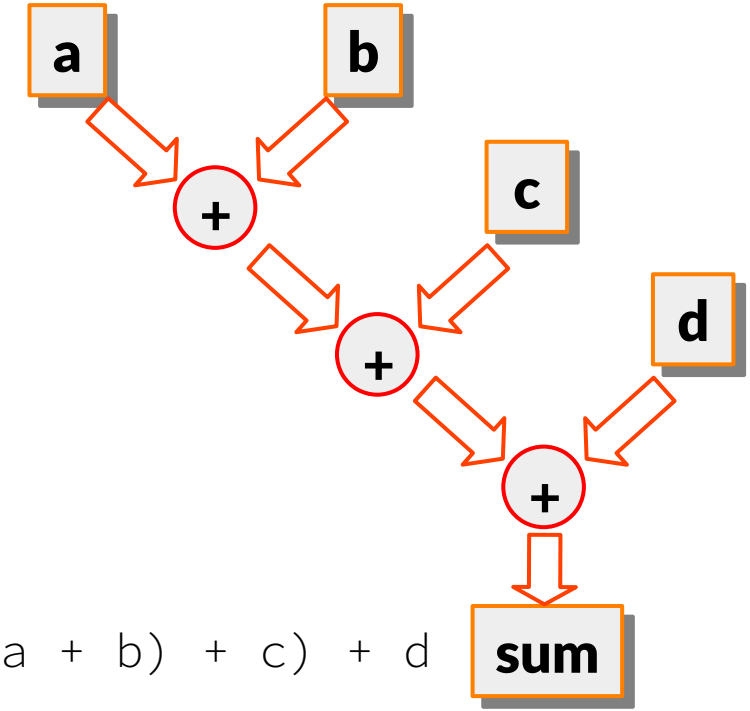
- Loading and storing is slow
  - Depends, but in the order of *10ns*
- Reading from registers is quick
  - Normally one clock cycle: *0.3-1ns*
  - Often hidden
- Reading data from registers *10-30x faster*
- Minimising loads/stores *absolutely critical* for high performance

Conflict!

- We don't have an infinite number of registers: 32 in RISC-V
- Some used by other parts of the program
- Some reserved for specific uses (more later)
- Why not more?
  - The law of diminishing returns
    - 1 → 2 registers > 2 → 4 > 4 → 8 > 8 → 16 > 16 → 32 > 32 → 64
  - More space in the instruction encoding
  - More silicon area
  - Slightly slower
- We need to do *as few load/stores* as possible, while using *as few registers* as possible

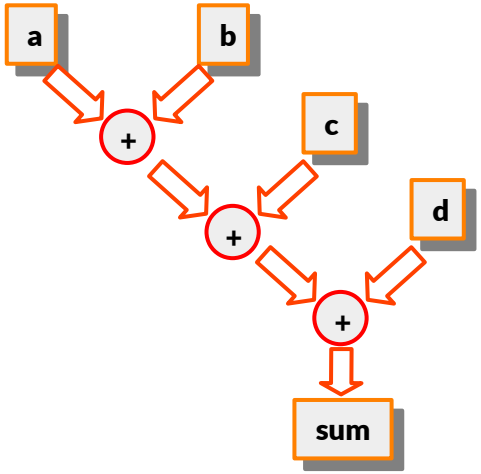
sum = a + b + c + d

- We need
  - 4 “lw” instructions
  - 3 “add” instructions
  - 1 “sw” instruction




```
sum = ((a + b) + c) + d
```

How many registers?



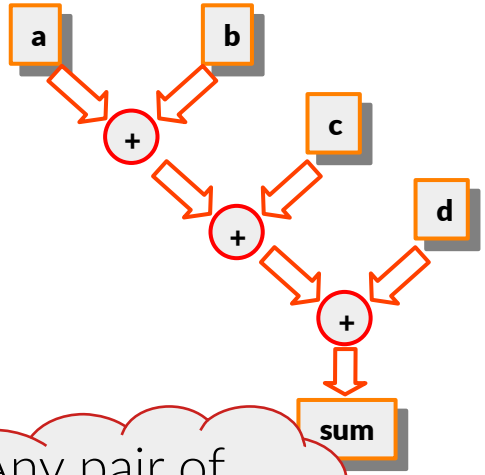
Version with 8 registers:

```
lw x1, a
lw x2, b
lw x3, c
lw x4, d
add x5, x1, x2 ;a+b
add x6, x5, x3 ;+c
add x7, x6, x4 ;+d
sw x7, sum, x8
```

Version with 2 registers:

```
lw x1, a
lw x2, b
add x1, x1, x2 ;a+b
lw x2, c
add x1, x1, x2 ;+c
lw x2, d
add x1, x1, x2 ;+d
sw x1, sum, x2
```

How many registers?



Version with 8 registers:

```
lw x1, a
lw x2, b
lw x3, c
lw x4, d
add x5, x1, x2 ;a+b
add x6, x5, x3 ;+c
add x7, x6, x4 ;+d
sw x7, sum, x8
```

Version with 2 registers:

```
lw x1, a
lw x2, b
add x1, x1, x2 ;a+b
lw x2, c
add x1, x1, x2 ;+c
lw x2, d
add x1, x1, x2 ;+d
sw x1, sum, x2
```

x1: accumulator

x2: temporary

Any pair of registers would work

Several statements

```
sum = ((a + b) + c) + d;
res = sum * a * b * c;
```

Version with 5 registers:

```
lw x1, a
lw x2, b
lw x3, c
lw x4, d
add x5, x1, x2 ;a+b
add x5, x5, x3 ;+c
add x5, x5, x4 ;+d
sw x5, sum, x4
mul x4, x5, x1 ;sum*a
mul x4, x4, x2 ;*b
mul x4, x4, x3 ;*c
sw x4, res, x1
```

First statement

Second statement

Several statements

```
sum = ((a + b) + c) + d;
res = sum * a * b * c;
```

Version with 5 registers:

```
lw x1, a
lw x2, b
lw x3, c
lw x4, d
add x5, x1, x2 ;a+b
add x5, x5, x3 ;+c
add x5, x5, x4 ;+d
sw x5, sum, x4
mul x4, x5, x1 ;sum*a
mul x4, x4, x2 ;*b
mul x4, x4, x3 ;*c
sw x4, res, x1
```

Version with 2 registers:

```
lw x1, a
lw x2, b
add x1, x1, x2 ;a+b
lw x2, c
add x1, x1, x2 ;+c
lw x2, d
add x1, x1, x2 ;+d
sw x1, sum, x2
lw x2, a
mul x1, x1, x2 ;sum*a
lw x2, b
mul x1, x1, x2 ;*b
lw x2, b
mul x1, x1, x2 ;*c
sw x1, res, x2
```

Several statements

```
sum = ((a + b) + c) + d;
res = sum * a * b * c;
```

Version with 5 registers:

```
lw x1, a
lw x2, b
lw x3, c
lw x4, d
add x5, x1, x2 ;a+b
add x5, x5, x3 ;+c
add x5, x5, x4 ;+d
sw x5, sum, x4
mul x4, x5, x1 ;sum*a
mul x4, x4, x2 ;*b
mul x4, x4, x3 ;*c
sw x4, res, x1
```

Version with 2 registers:

```
lw x1, a
lw x2, b
add x1, x1, x2 ;a+b
lw x2, c
add x1, x1, x2 ;+c
lw x2, d
add x1, x1, x2 ;+d
sw x1, sum, x2
lw x2, a
mul x1, x1, x2 ;sum*a
lw x2, b
mul x1, x1, x2 ;*b
lw x2, b
mul x1, x1, x2 ;*c
sw x1, res, x2
```

Which one is better?

- Number of instructions?
- Number of registers?
- Number of memory operations?

Not always obvious

Reusing variables

- Short sequences
  - Fewer registers is usually as fast
- Longer sequences that reuse variables
  - More registers is better
- Extra loads are bad
  - More instructions
  - More time
  - More energy!
- But context! Minimising sequence loads, does not always minimise total loads

Another example

```
res = (a + b) * (c + d) * (a + c)
```

"Natural" Order

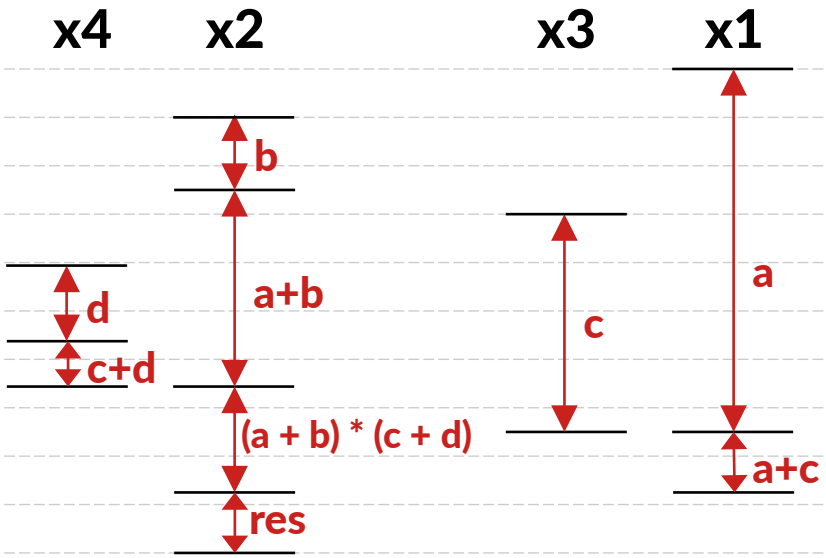
```
lw x1, a
lw x2, b
add x2, x1, x2 ;a+b
lw x3, c
lw x4, d
add x4, x3, x4 ;c+d
mul x2, x2, x4 ;(a+b)*(c+d)
add x1, x1, x3 ;a+c
mul x2, x2, x1 ;res
sw x2, res, x1
```

Another example

`res = (a + b) * (c + d) * (a + c)`

"Natural" Order

```
lw x1, a
lw x2, b
add x2, x1, x2 ;a+b
lw x3, c
lw x4, d
add x4, x3, x4 ;c+d
mul x2, x2, x4 ;(a+b)*(c+d)
add x1, x1, x3 ;a+c
mul x2, x2, x1 ;res
sw x2, res, x1
```



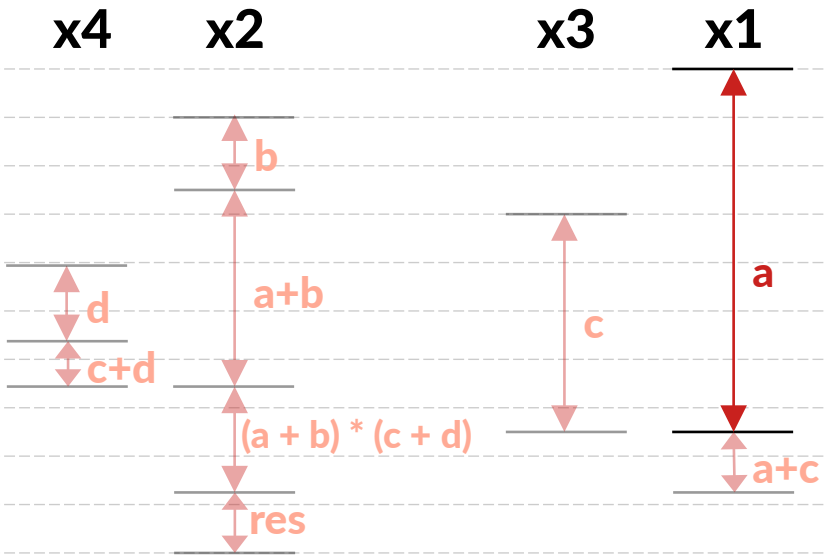
Value Liveness

Another example

`res = (a + b) * (c + d) * (a + c)`

"Natural" Order

```
lw x1, a
lw x2, b
add x2, x1, x2 ;a+b
lw x3, c
lw x4, d
add x4, x3, x4 ;c+d
mul x2, x2, x4 ;(a+b)*(c+d)
add x1, x1, x3 ;a+c
mul x2, x2, x1 ;res
sw x2, res, x1
```



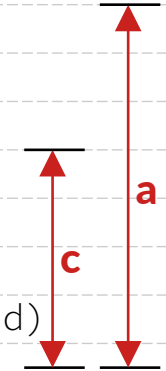
Value Liveness

Another example

```
res = (a + b) * (c + d) * (a + c)
```

"Natural" Order

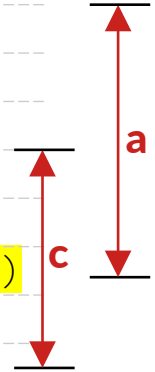
```
lw x1, a
lw x2, b
add x2, x1, x2 ;a+b
lw x3, c
lw x4, d
add x4, x3, x4 ;c+d
mul x2, x2, x4 ;(a+b)*(c+d)
add x1, x1, x3 ;a+c
mul x2, x2, x1 ;res
sw x2, res, x1
```



Four registers

Optimised Order

```
lw x1, a
lw x2, b
add x2, x1, x2 ;a+b
lw x3, c
add x1, x1, x3 ;a+c
mul x1, x1, x2 ;(a+b)*(a+c)
lw x2, d
add x2, x3, x2 ;c+d
mul x1, x1, x2 ;res
sw x1, res, x2
```



Three registers

One last example

```
res = a - b + c * d
```

"Natural" Order

```
lw x1, a
lw x2, b
sub x1, x1, x2 ;a-b
lw x2, c
lw x3, d
mul x2, x2, x3 ;c*d
add x1, x1, x2 ;sum
sw x1, res, x2
```

Three registers

Optimised Order

```
lw x1, c
lw x2, d
mul x1, x1, x2 ;c*d
lw x2, a
add x1, x1, x2 ;a+c*d
lw x2, b
sub x1, x1, x2
sw x1, res, x2
```

Two registers

Re-arranging expressions: Caution ⚡

- Moving instructions is legal **if and only if** the modified sequence produces the exact same result
  - Integer arithmetic: overflow errors
  - a – b + c
  - a = 200, b = 180, c = 100, 8-bit registers (max value = 255)
  - Original: `(a – b) + c = (200 – 180) + 100 = 20 + 100 = 120`
  - Modified: `(a + c) – b = (200 + 100) – 180 = 44 – 180 = -136`

Re-arranging expressions: Caution ⚡

- Moving instructions is legal **if and only if** the modified sequence produces the exact same result
  - Floating point arithmetic: non-associative & non-exact
    - `a + b` might be different than `b + a`
  - Integer division: rounding errors
    - `(3 / 2) + (5 / 2) = 3`
    - `(3 + 5) / 2 = 4`

Instruction scheduling & Register allocation

- Do not necessarily make code faster
  - But might make surrounding code faster!
- Hard problem even for short code sequences
  - Optimal solution depends on context
  - NP-complete problem → optimal solution infeasible for large blocks / whole program
  - Millions of lines of compiler code has been written

Compilers are better than you

(only in terms of translating high level code to assembly)

Summary

- Three conflicting goals
  - Minimise instructions
  - Minimise memory accesses
  - Minimise register use
- Generally reducing memory use is the most critical one
  - Memory wall
- Reorder statements to optimise code
  - As long as the result is the same

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Control Constructs

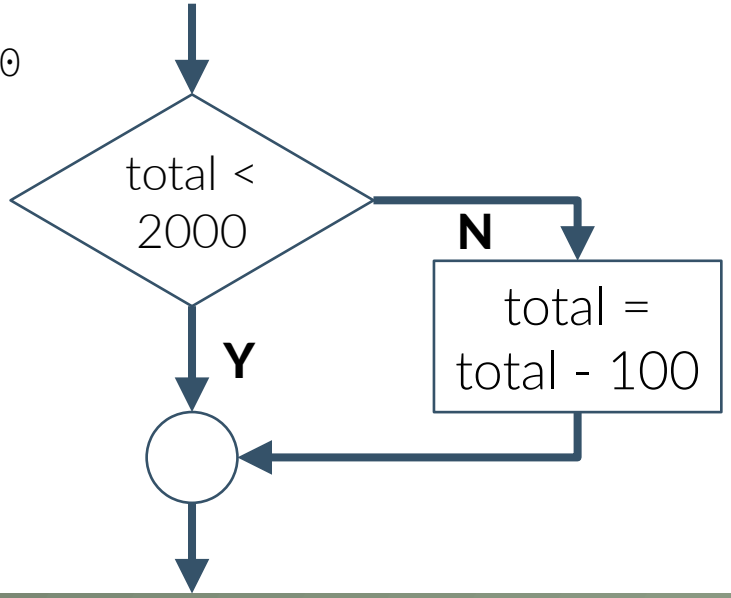
Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- From Python/Java/C to RISC-V code
  - if …
  - if … else …
  - for …
  - while …

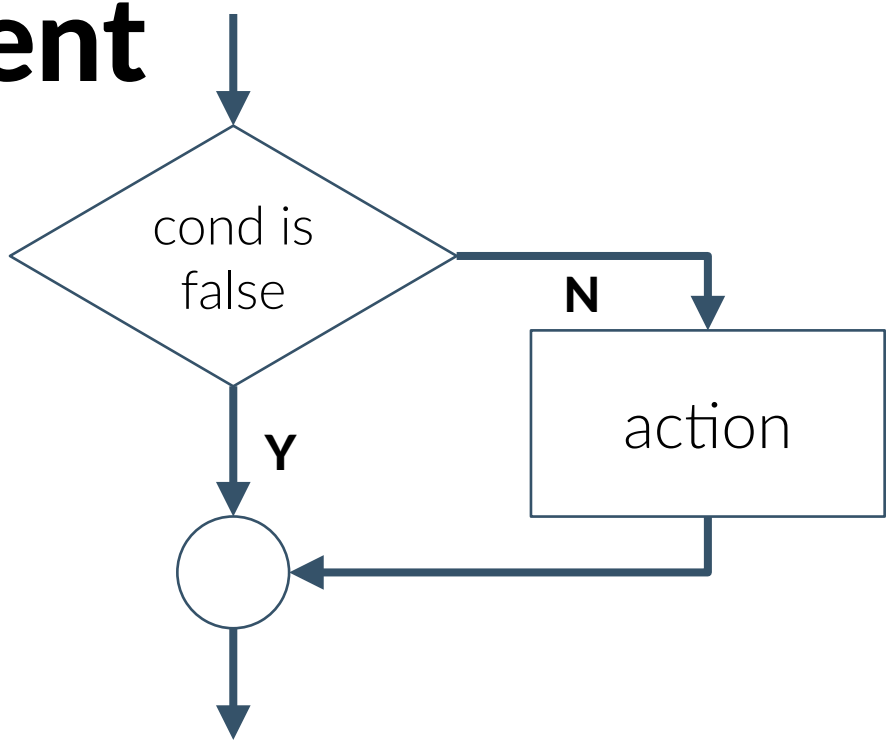
Earlier this semester

- Textual description
  - if total ≥ 2000 pounds then deduct 100 pound from total
- Python
  - ```
    if total >= 2000:
        total = total - 100
    ```
- Assembly


    ```

Generalised if-statement

- `if cond:`
  `action`



If vs branch

- Branch
  - Program executes something else when true
  - Program executes following piece of code when false
- if
  - Following piece of code executes when true
  - Following code does not execute when false

Branches designed to skip actions

```
    blt x1, x2, skip
    action_block
skip
    ...
```

```
    bge x1, x2, action
    j skip
action
    action_block
skip
    ...
```

RISC-V ↔ if statement

- `if <statement1> <cond> <statement2>:`
  `action`
  - Evaluate statement1 → rs1
  - Evaluate statement2 → rs2
  - `b<opposite cond> rs1, rs2, skip_action`
  - action

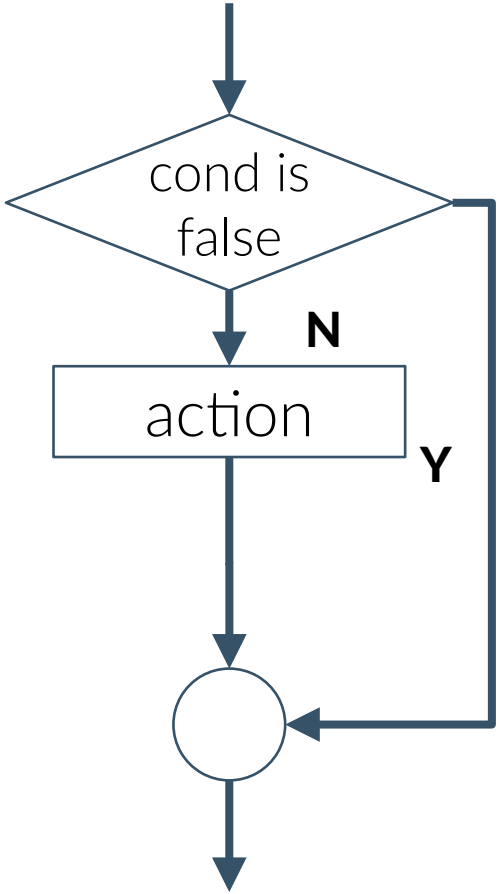
Example: if

- `if a + b >= c * d:`
  `    e = f`

```
    lw x1, a
    lw x2, b
    add x1, x1, x2 ; a + b
    lw x2, c
    lw x3, d
    mul x2, x2, x3; c * d
    blt x1, x2, skip
    lw x1, f
    sw x1, e, x2
skip
    …
```

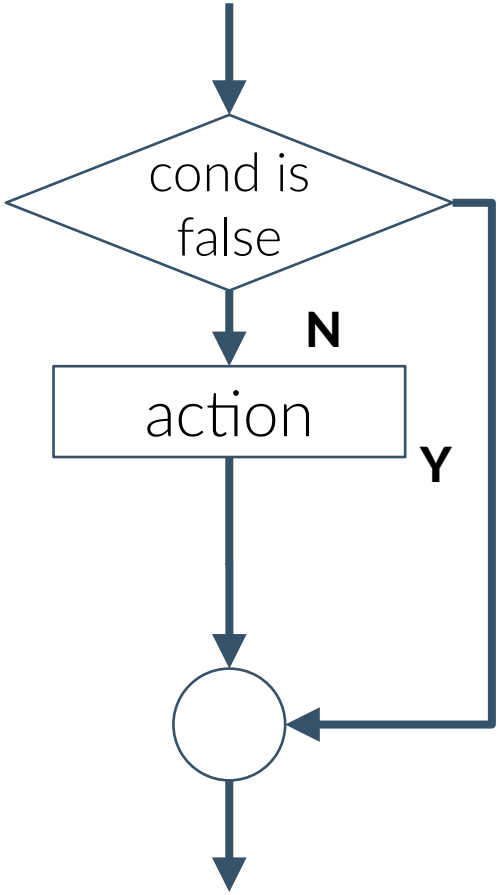
Generalised if-statement

- `if cond:`
  `action`



Generalised if-else-statement

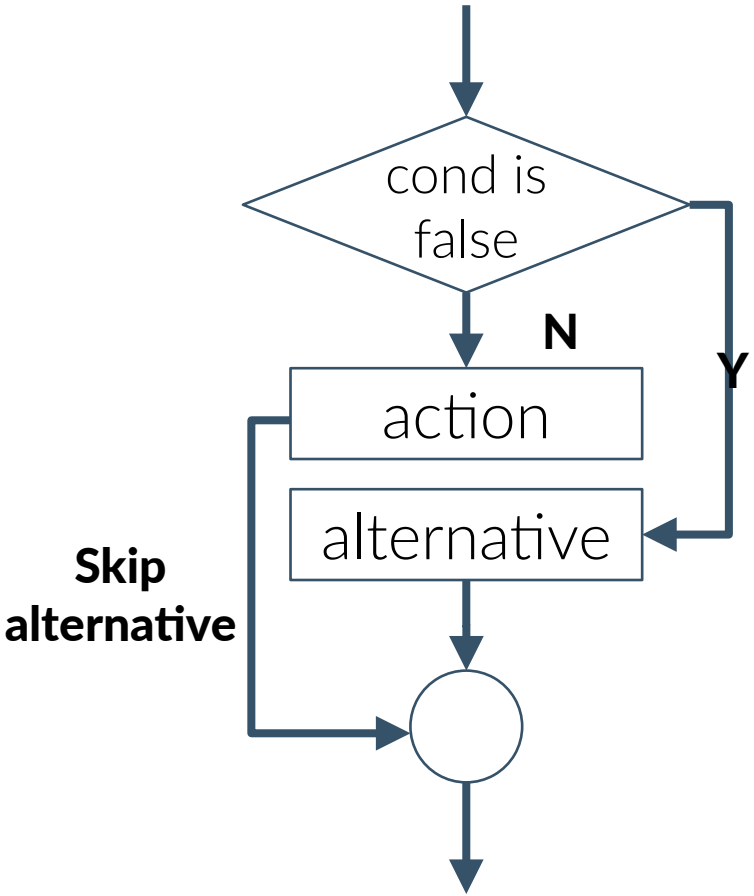
-
```
if cond:
    action
else:
    alternative
```



Generalised if-else-statement

-
```
if cond:
    action
else:
    alternative
```

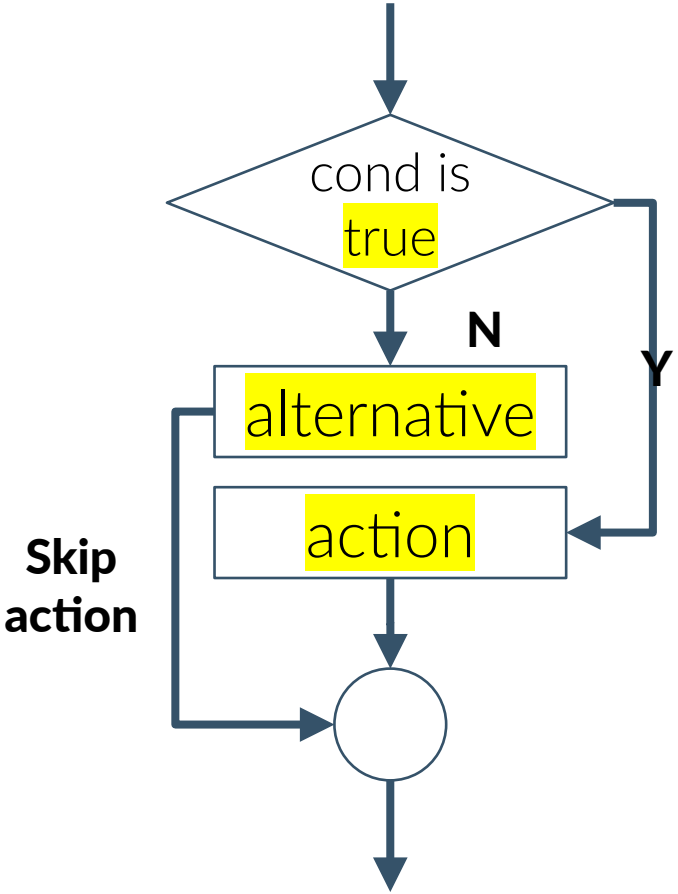
Why still reversing the condition?



Generalised if-else-statement

- ```
  if cond:
      action
  else:
      alternative
  ```

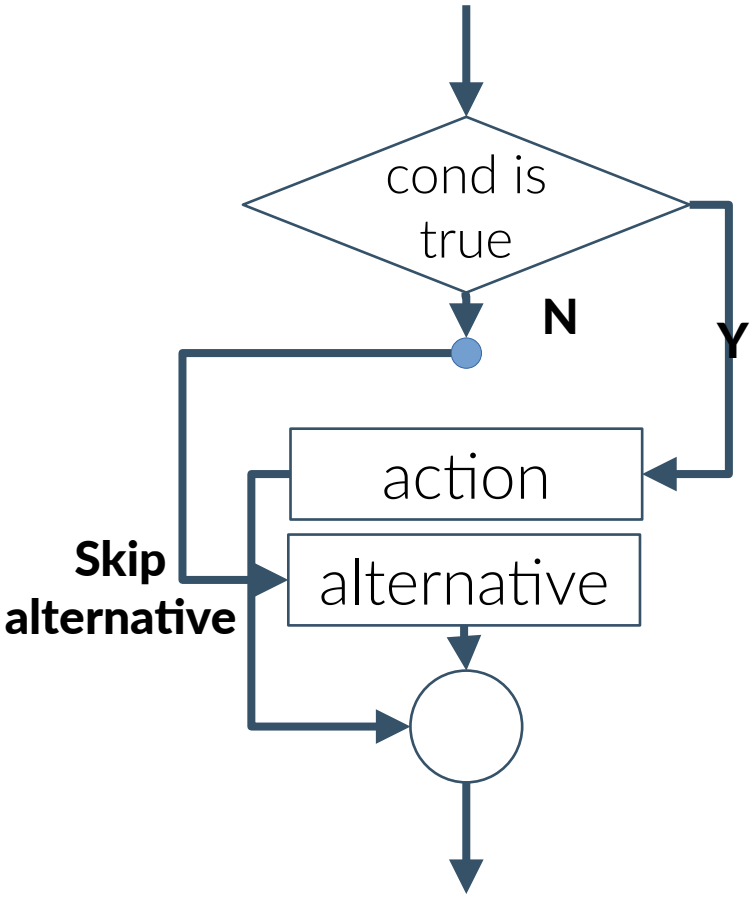
Original condition and reverse block order is OK


  ```

Generalised if-else-statement

-
```
if cond:
    action
else:
    alternative
```

Original condition and block order is not



RISC-V ↔ if-else statement

- ```
  if <statement1> <cond> <statement2>:
      action
  else
      alternative
  ```

  - Evaluate statement1 → rs1
  - Evaluate statement2 → rs2
  - `b<opposite cond> rs1, rs2, skip_action`
  - action
  - `j skip_alternative`
  - alternative
  ```

Example: if-else

- ```
  if a == b:
      c = d
  else:
      c = e
  ```

Can you optimise this?

```
    lw x1, a
    lw x2, b
    bne x1, x2, skip_action
    lw x1, d
    sw x1, c, x2
    j skip_alternative
skip_action
    lw x1, e
    sw x1, c, x2
skip_alternative
    ...
```
```

Example: if-else

```
    lw x1, a
    lw x2, b
    bne x1, x2, skip_action
    lw x1, d
    sw x1, c, x2
    j skip_alternative
skip_action
    lw x1, e
    sw x1, c, x2
skip_alternative
    ...
```

```
    lw x1, a
    lw x2, b
    bne x1, x2, skip_action
    lw x1, d
    j skip_alternative
skip_action
    lw x1, e
skip_alternative
    sw x1, c, x2
    ...
```

Example: if-else

```
    lw x1, a
    lw x2, b
    bne x1, x2, skip_action
    lw x1, d
    sw x1, c, x2
    j skip_alternative
skip_action
    lw x1, e
    sw x1, c, x2
skip_alternative
    ...
```

```
    lw x1, a
    lw x2, b
    lw x3, e
    bne x1, x2, skip_action
    lw x3, d
    j skip_alternative
skip_action
    lw x1, e
skip_alternative
    sw x3, c, x2
    ...
```

Example: if-else

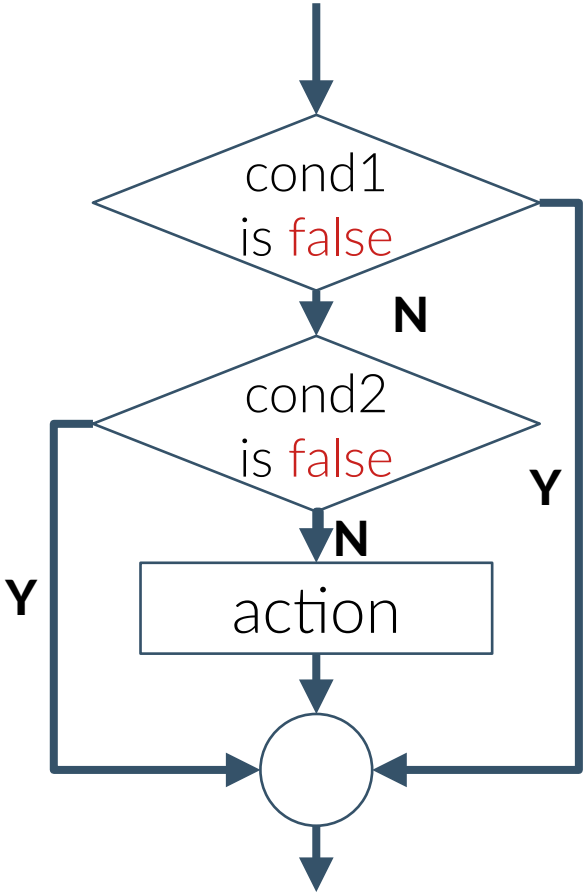
8 vs 7 instructions
2 vs 1 control flow

```
    lw x1, a
    lw x2, b
    bne x1, x2, skip_action
    lw x1, d
    sw x1, c, x2
    j skip_alternative
skip_action
    lw x1, e
    sw x1, c, x2
skip_alternative
    ...
```

```
    lw x1, a
    lw x2, b
    la x3, e
    bne x1, x2, skip_action
    la x3, d
    j skip_alternative
skip_action
    lw x1, e
skip_alternative
    lw x3, 0[x3]
    sw x3, c, x2
    ...
```

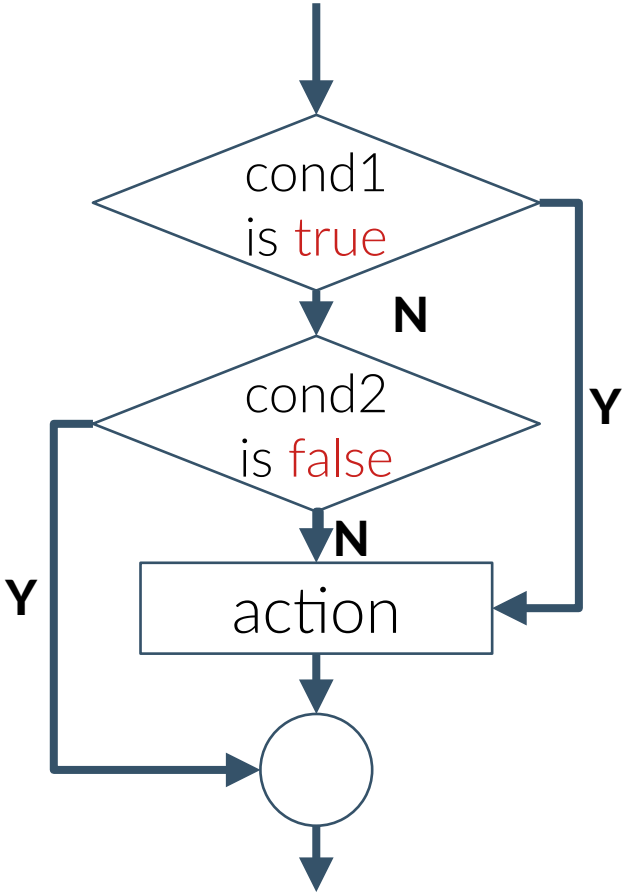
and-composite if statement

- ```
  if cond1 and cond2:
      action
  ```
- ```
  if cond1:
      if cond2:
          action
  ```



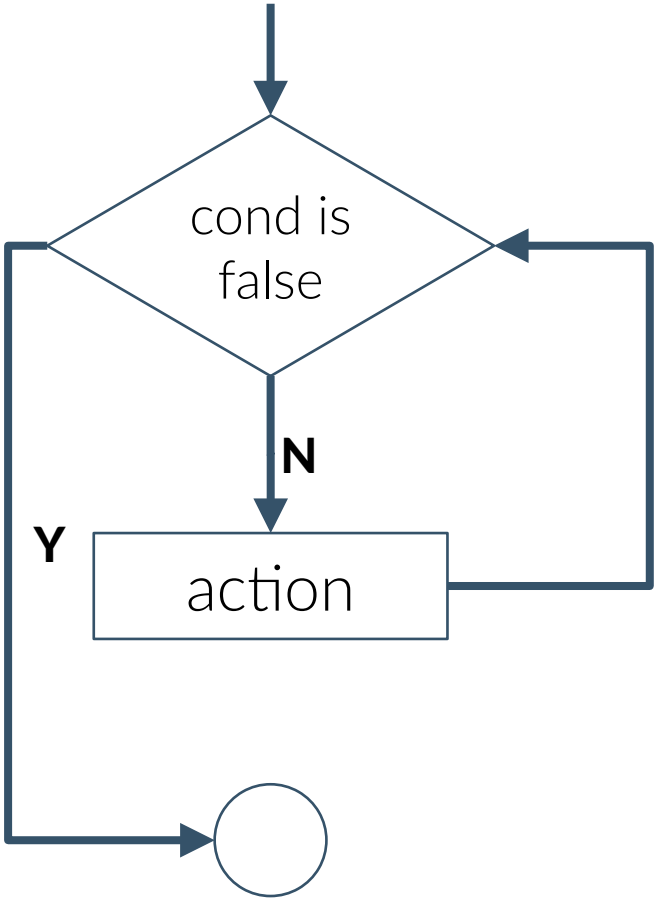
or-composite if statement

- ```
  if cond1 or cond2:
      action
  ```
- ```
  if cond1:
      action
  else if cond2:
      action
  ```
- ```
  if not cond1:
      if cond2:
          action
  else:
      action
  ```

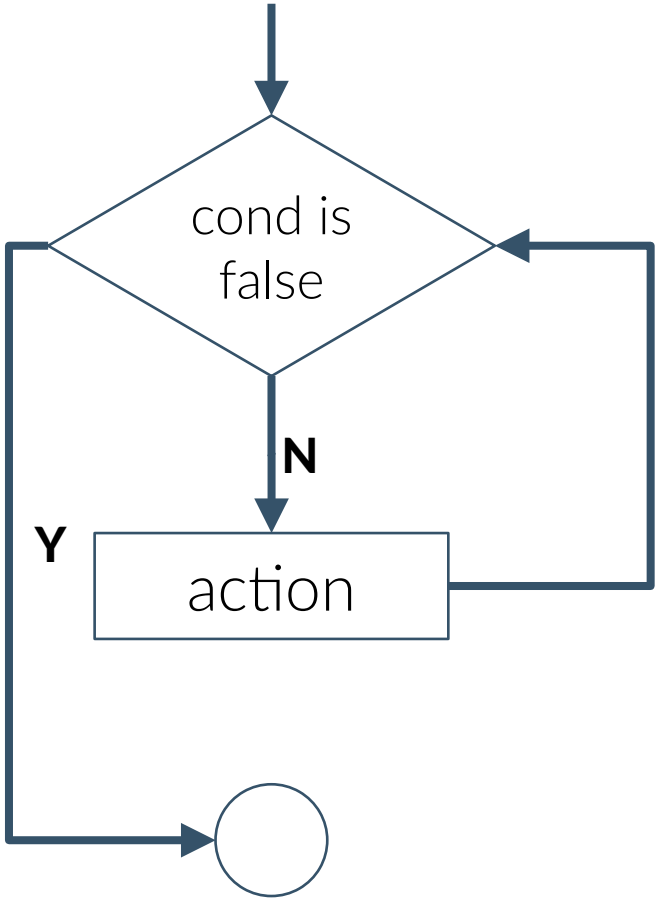

  ```

Generalised while-statement

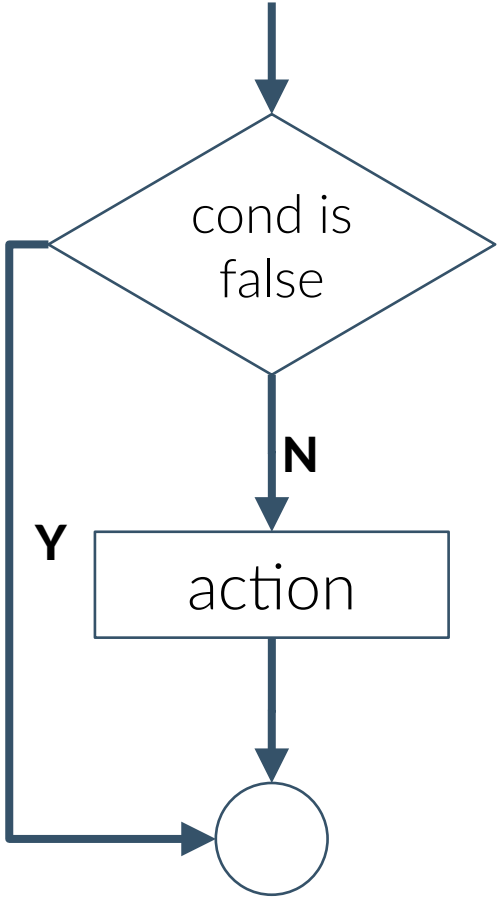
- `while cond:`
  `action`



while-stmt



if-stmt



RISC-V ↔ while statement

- `while <statement1> <cond> <statement2>:`
  `action`
  - Evaluate statement1 → rs1
  - Evaluate statement2 → rs2
  - `b<opposite cond> rs1, rs2, exit_loop`
  - action
  - `j loop_start`

Example: while

- `while a < b:`
  `a = a * 2`

```
start
    lw x1, a
    lw x2, b
    bge x1, x2, skip
    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    j start
skip
    …
```

Blindly transcribing high level code to assembly is inefficient!

Summary

- High level control flow to low level control flow
  - if/if-else/while
- Comparison sub-expressions → register values
- Comparison → inverse branch condition
- Composite if statements → chain of branches
- Efficient high level code != efficient low level code

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Optimising Loops

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- Loops critical for performance
  - They're everywhere
  - Small sequences → executed repeatedly
  - Small changes → massive effect
- Hardware tuned for making loop-y code faster
  - Software needs to match what the hardware is tuned for

Example: while

- `while a < b:`
  `a = a * 2`

2 control flow instrs per iteration
1 has no effect
(except for the last iteration)

```
start
    lw x1, a
    lw x2, b
    bge x1, x2, skip
    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    j start
skip
    …
```

Blindly transcribing high level code to assembly is inefficient!

- `while a < b:`
  `a = a * 2`

Original

```
start
    lw x1, a
    lw x2, b
    bge x1, x2, skip
    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    j start
skip
    …
```

This will run once even if a > b

Optimised

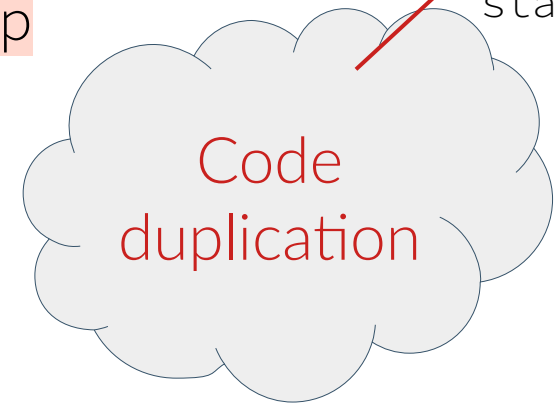
```
start
    lw x1, a
    lw x2, b

    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    blt x1, x2, start
skip
    …
```

- `while a < b:`
  `a = a * 2`

Original

```
start
    lw x1, a
    lw x2, b
    bge x1, x2, skip
    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    j start
skip
    …
```



Optimised

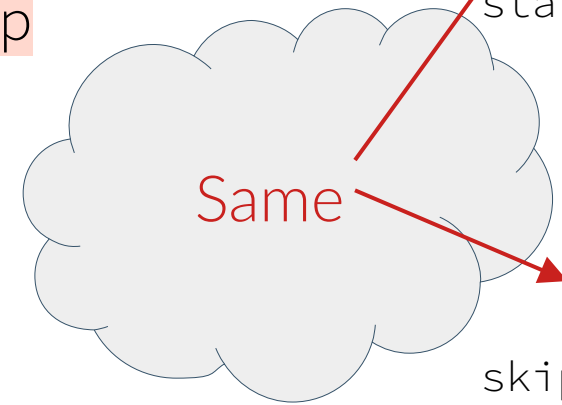
```
prestart
    lw x1, a
    lw x2, b
    bge x1, x2, skip
start
    lw x1, a
    lw x2, b

    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    blt x1, x2, start
skip
    …
```

- `while a < b:`
  `a = a * 2`

Original

```
start
    lw x1, a
    lw x2, b
    bge x1, x2, skip
    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    j start
skip
    …
```



Optimised

```
prestart
    lw x1, a
    lw x2, b
    bge x1, x2, skip
start
    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    lw x1, a
    lw x2, b
    blt x1, x2, start
skip
    …
```

- `while a < b:`
  `a = a * 2`

Original

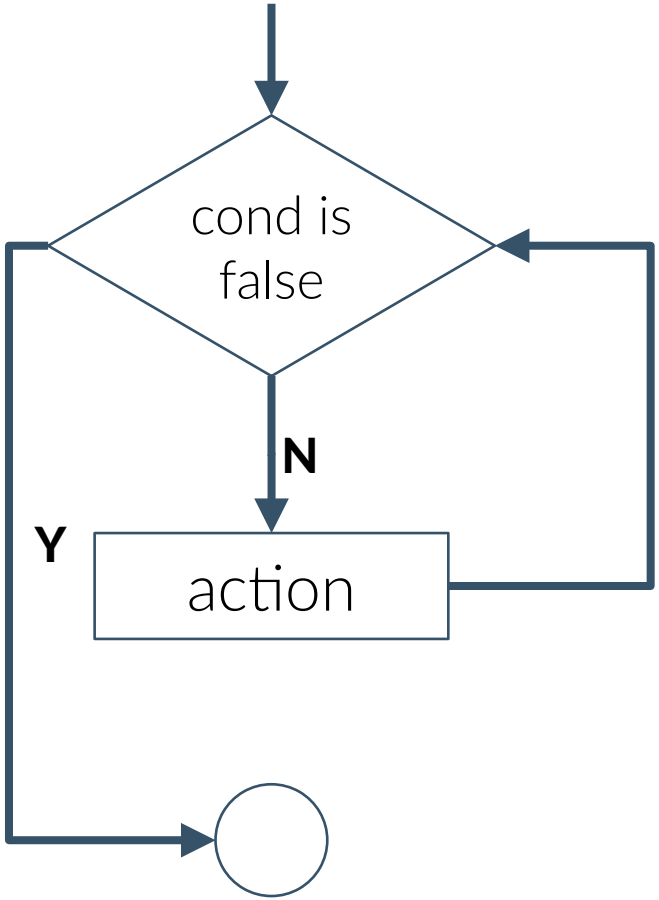
```
start
    lw x1, a
    lw x2, b
    bge x1, x2, s
    li x2, 2
    mul x1, x1, x
    sw x1, a, x2
    j start
skip
    …
```

Moved one expensive instruction out of the loop

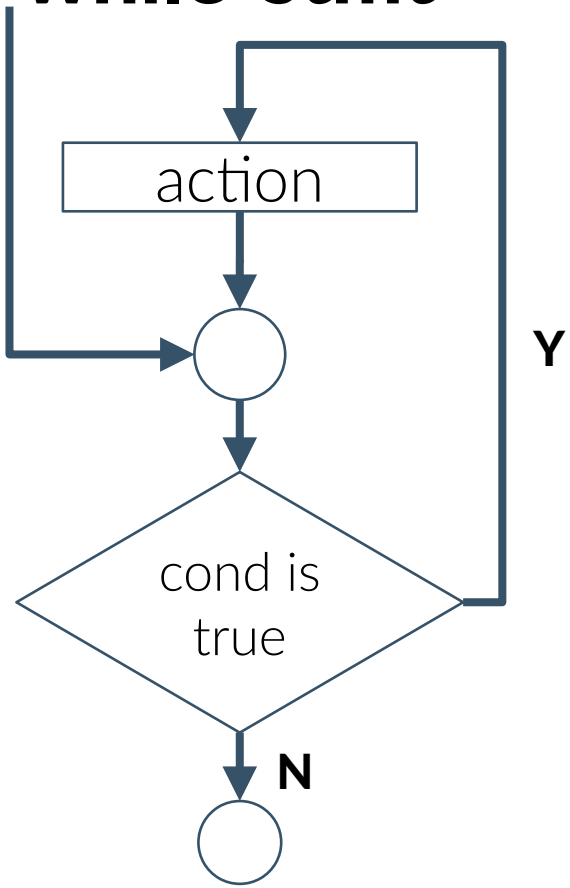
Optimised

```
    j start
loop
    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
start
    lw x1, a
    lw x2, b
    blt x1, x2, loop
skip
    …
```

while-stmt



Modified while-stmt



Loop Invariant Code Motion

Original

```
start
    lw x1, a
    lw x2, b
    bge x1, x2, skip
    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    j start
skip
    …
```

b

2

b never changes

2 never changes

Optimised

```
    lw x2, b
    li x2, 2
start
    lw x1, a
    bge x1, x2, skip
    mul x1, x1, x2
    sw x1, a, x2
    j start
skip
    …
```

b

2

Loop Invariant Code Motion

Original

```
start
    lw x1, a
    lw x2, b
    bge x1, x2, sk
    li x2, 2
    mul x1, x1, x
    sw x1, a, x2
    j start
skip
    …
```

b

2

1 lw less
1 li less
2 extra
registers!

Optimised

```
    lw x2, b
    li x3, 2
tart
    lw x1, a
    bge x1, x2, skip
    mul x1, x1, x3
    sw x1, a, x4
    j start
skip
    …
```

b

2

Loop Invariant Code Motion

Original

```
start
    lw x1, a
    lw x2, b
    bge x1, x2, skip
    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    j start
skip
    …
```

Reading/ writing the same thing

Optimised

```
    lw x2, b
    li x3, 2
start
    lw x1, a
    bge x1, x2, skip
    mul x1, x1, x3
    sw x1, a, x4
    j start
skip
    …
```

Register Promotion

Original

```
start
    lw x1, a
    lw x2, b
    bge x1, x2, sk
    li x2, 2
    mul x1, x1, x
    sw x1, a, x2
    j start
skip
    …
```

1 lw less
1 register less

Optimised

```
    lw x2, b
    li x3, 2
    lw x1, a
tart
    bge x1, x2, skip
    mul x1, x1, x3
    j start
skip
    sw x1, a, x2
    …
```

Change branching logic

Original

```
start
    lw x1, a
    lw x2, b
    bge x1, x2, skip
    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    j start
skip
    …
```

Optimised

```
    lw x2, b
    li x3, 2
    lw x1, a
    j start
loop
    mul x1, x1, x3
start
    blt x1, x2, loop
skip
    sw x1, a, x2
    …
```

Strength Reduction

Original

```
start
    lw x1, a
    lw x2, b
    bge x1, x2, skip
    li x2, 2
    mul x1, x1, x2
    sw x1, a, x2
    j start
skip
    …
```

7 vs 2 loop instructions
No memory
No multiplication

Optimised

```
    lw x2, b
    li x3, 2
    lw x1, a
    j start
loop
    add x1, x1, x1
start
    blt x1, x2, loop
skip
    sw x1, a, x2
    …
```

For loops

- for i in range(0, 10):
  action
- i = 0
  while i < 10:
  action
  i = i + 1

Count from zero to something

For loops

- for i in range(0, 10):
  action
- i = 0
  while i < 10:
  action
  i = i + 1

```
li x1, 0
li x2, 10
start
   action
   addi x1, x1, 1
   blt x1, x2, start
skip
```

- for i in range(9, -1, -1):
  action
- i = 9
  while i > -1:
  action
  i = i -1

For loops

- for i in range(0, 10):
  action
- i = 0
  while i < 10:
  action
  i = i + 1

```
li x1, 0
li x2, 10
start
   action
   addi x1, x1, 1
   blt x1, x2, start
skip
```

- for i in range(9, -1, -1):
  action
- i = 9
  while i >= 0:
  action
  i = i -1

```
li x1, 9

start
   action
   addi x1, x1, -1
   bge x1, x0, start
skip
```

For loops

- for i in range(0, 10):
  action

- i = 0
  while i < 10:
  action
  i = i + 1

```
li x1, 0
li x2, 10
start
    action
    addi x1, x1, 1
    blt x1, x2, start
skip
```

- for i in range(9, -1, -1):
  action

- i = 9
  while i >= 0:
  action
  i = i -1

```
li x1, 0

start
    action
    addi x1, x1, -1
    bgez x1, start
skip
```

1 register less
Likely to be
optimised by
HW

For loops

- for i in range(0
  action
- i = 0
  while i
  actio
  i = i

```
li x1,
li x2,
start
    actio
    addi x1, x                    , -1
    blt x1, x2,                   x1, start
skip
```

ONLY IF EXECUTION ORDER DOES NOT MATTER!

Summary

- Optimising loops is critical for performance
- Conditional branch at the end of the loop
- Loop Invariant Code Motion
- Register Promotion
- Strength Reduction
- Count down to zero